



< Previous



Next >

Prefix codes

🔖 Bookmark this page

Problem: What do prefix Turing machines look like?

A prefix Turing machine is any Turing machine that only halts for prefix programs (*i.e.* programs with an unambiguous end).

The binary codes of these program constitute a **prefix code**. This means that no code word in this set is the beginning of another code word. Prefix codes are **uniquely decodable**, which means that there is no ambiguity when interpreting the code (note that the converse is not true: {01, 011, 0111} is perfectly decodable, though it is not prefix).

Prefix codes

1/1 point (ungraded)

A code with the codewords {01, 001, 0001, 10, 110, 1110} is:

- ☐ Not prefix but uniquely decodable
- ☐ Not prefix and not uniquely decodable
- ☒ Prefix and uniquely decodable
- ☐ Prefix but not uniquely decodable

✓
Answer
Correct:
None of these code words is the continuation of another one. The code is prefix and uniquely decodable.

Submit You have used 1 of 1 attempt

Answers are displayed within the problem

Interestingly, Huffman codes are designed to be prefix.
One way for programs to be prefix is to be **self-delimited**.

3	0111
4	01111

les

g a string *s* in a self-
ts of duplicating each
e coded string is
he doubling rule,
te a 0 or 10 to

: of 1268:

110001

Decimal to binary converter:

117

Convert

(117)₁₀ = (1110101)₂

Binary to decimal converter:

Convert

Doubling length Code 1

1/1 point (ungraded)

What is the "doubling length" code for 117 ?

- ☐ 11001010001
- ☒ 111101110101
- ☐ 1111100110010
- ☐ 110011010001011101

✓

Answer
Correct:
117 --> 1111101110101
where 1110101 is the binary code for 117, and 111110 is the double code for its length (7).

Submit

You have used 1 of 1 attempt

Answers are displayed within the problem

Doubling length Code 2

1/1 point (ungraded)

Open PrefixInt and locate the function DoublingLengthCode that is meant to code integers using the doubling length code. Paste the correct "return" line into the answer box.

return CrudeDoublingCode(len(BN))+BN

✓ Answer: .+CrudeDoublingCode.\s*len.\s*BN\s*.\s*.\s*\+.\s*BN

```
def DoublingLengthCode(N):  
    BN = Binary(N)  
    return DoublingCode(len(BN)) + BN
```

Submit

You have used 1 of 1 attempt

Answers are displayed within the problem

Decoding Doubling Length Code 1

0/1 point (ungraded)

In PrefixInt, locate the function DoublingLengthDecode . Fill-in the blanks in the line
StrN = BinN[.... :]
and paste this line only in the answer box.

StrN = BinN[2*len(BinLength):]



? Hint (1 of 1):

StrN = BinN[2*len(BinLength) : 2*len(BinLength)+Length]

Next Hint

Submit

You have used 1 of 1 attempt

✖ Incorrect (0/1 point)

Decoding Doubling Length Code 2

0/1 point (ungraded)

You may now use the `DoublingLengthDecode` function.

```
$ python
>>> import PrefixInt as PI
>>> PI.DoublingLengthDecode('11000000101111000100100000001')
```

Can you decode: `11000000101111000100100000001` ?

- ☐ 11117
- ☐ 482
- ☐ 10000
- ☐ 4242
- ☒ 123456

✓
- ☒ 493825



Answer
Incorrect:

```
python
>>> import PrefixInt as PI
>>> PI.DoublingLengthDecode('11000000101111000100100000001')
{'Number': 123456, 'Length': 17, 'Remainder': '01'}
```

Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem



edX

- About
- Affiliates
- edX for Business
- Open edX
- Careers
- News

Legal

- Terms of Service & Honor Code
- Privacy Policy
- Accessibility Policy
- Trademark Policy
- Sitemap
- Cookie Policy
- Your Privacy Choices

Connect

- Idea Hub
- Contact Us
- Help Center
- Security
- Media Kit

